

Relatório do Exercício-Programa de Etiquetagem Morfossintática (EP1)

Aluno: *[Laufernando Souza Dias - 11222947]*

Disciplina: Introdução ao Processamento de Língua Natural - MAC0508

28 de outubro de 2025

1 Descrição da Implementação

A implementação foi realizada em ambiente **Linux Fedora 6.17.4-200.fc42.x86_64**, com processador **Intel(R) Core(TM) i5-13420H** e 8 GB de RAM.

O exercício foi dividido em duas partes principais: **Parte A**, referente ao modelo de bigramas para predição de palavras, e **Parte B**, referente ao modelo **HMM** (Hidden Markov Model) com treinamento pelo algoritmo de *Baum-Welch* e etiquetagem por *Viterbi*.

Filtragem e Pré-processamento

A etapa de filtragem é responsável por carregar e limpar os dados no formato CoNLL-U, utilizando a biblioteca `conllu`. São removidos tokens que representam *multiwords*, e todas as palavras são convertidas para minúsculas, garantindo padronização entre os conjuntos de treino, validação e teste.

Essas operações são realizadas antes das partes A e B, e garantem que apenas os tokens relevantes sejam processados.

Parte A: Modelo de Bigramas

O modelo de bigramas foi implementado na classe `BigramModel`, que utiliza uma estrutura esparsa (`defaultdict(Counter)`) para armazenar contagens de unigramas e bigramas, aplicando suavização de Lidstone ($\lambda = 1.0$).

Para cada sentença, adicionam-se marcadores de início e fim (`<s>` e `</s>`), e o modelo prevê a próxima palavra a partir da atual. A avaliação considera a acurácia global e também métricas por palavra.

Apesar de funcionar corretamente, o modelo atingiu apenas **18% de acurácia**, mesmo após testes com diferentes valores de λ_{smooth} . Isso se deve ao fato de o modelo capturar

apenas dependências locais entre palavras, sem considerar contexto sintático ou morfológico.

Códigos adicionais de teste. Foram comentadas no código funções experimentais para avaliar *top-k accuracy* e métricas por palavra (precisão, revocação e F1). Por exemplo, a função `top_k_accuracy(k)` permite medir quantas vezes a palavra correta aparece entre as k mais prováveis. Um caso percebido por mim foi o da palavra “latina”, que apresentou *recall* 1.0 mas *precision* 0.57, indicando generalização incorreta em contextos novos. Ao que parece, no conjunto de treinamento, “latina” aparece frequentemente após “américa”, mas no conjunto de teste, “latina” pode aparecer em contextos diferentes, levando o modelo a cometer erros.

Exemplo de saída:

Top 20 palavras com melhor recall:

palavra precision recall f1 suporte

(...)

39 latina 0.571429 1.0 0.727273 4

Parte B: Modelo HMM

A Parte B implementa um modelo HMM de segunda ordem para etiquetagem morfosintática, onde cada estado é formado por um par de tags (t_{i-1}, t_i) . A classe `HMM` encapsula a criação das matrizes de transição (A), emissão (B) e distribuição inicial (π).

O treinamento é feito pelo algoritmo de *Baum-Welch*, usando apenas `NumPy`. Devido à ausência de processamento paralelo e limitações de hardware, o corpus de treino foi dividido em lotes (*batches*) de 100 sentenças, e a cada época o usuário pode optar por continuar o treinamento. A convergência é verificada pela distância entre matrizes sucessivas.

A cada época, o tempo de execução foi de aproximadamente **50 minutos** sobre o corpus completo (5893 sentenças), com redução da distância de 5.88 para 4.10 após duas épocas, e não consegui treinar por mais tempo. O critério de parada usa $\varepsilon = 10^{-3}$ tanto para cálculos diretos quanto em log-space (ε_{log}).

2 Execução do Programa

A execução principal é feita via terminal, no diretório que contém todos os arquivos dependentes (`parteA.py`, `parteB.py`, `filtragem.py`, e os arquivos `.conllu`). Também podem existir arquivos `.csv` com o modelo HMM exportado.

```
$ python main.py
```

Durante a execução, o programa pergunta se deve carregar um modelo existente, treinar um novo e exportar os resultados. A cada época de treino, o usuário é informado do progresso e da distância atual.

Por padrão, o prefixo dos arquivos de modelo é `hmm_iter`. As matrizes A , B e o vetor π podem ser exportados ou importados em formato CSV.

3 Resultados

Parte A – Bigramas

- Acurácia: 18%
- Observação: Mesmo com suavização, o modelo não generaliza bem para palavras fora de contexto frequente.

Parte B – HMM com Viterbi

Após o treinamento e a etiquetagem do corpus de teste, obtiveram-se os seguintes resultados:

```
=====
RELATÓRIO DE AVALIAÇÃO
=====
```

Acurácia: 0.9214 (92.14%)

Acertos: 30952/33594

Macro Average:

Precision: 0.8763

Recall: 0.8219

F1: 0.8397

F1 (P+R): 0.8482

Weighted Average:

Precision: 0.9205

Recall: 0.9214

F1: 0.9187

Métricas por tag (top 5 piores F1):

X : P=0.591 R=0.213 F1=0.313 (n=61)

INTJ : P=0.667 R=0.400 F1=0.500 (n=5)
SCONJ : P=0.784 R=0.717 F1=0.749 (n=456)
PROPN : P=0.917 R=0.666 F1=0.772 (n=2209)
ADJ : P=0.832 R=0.796 F1=0.813 (n=1676)

=====

Os resultados demonstram desempenho robusto do modelo HMM, atingindo **92% de acurácia global**, com métricas equilibradas entre precisão e recall. As categorias mais desafiadoras incluem X e INTJ, devido à baixa frequência no corpus e maior ambiguidade.

4 Comentários e Conclusões

O exercício permitiu compreender de forma prática a diferença entre abordagens baseadas em n-gramas e modelos probabilísticos de estados ocultos. O modelo de bigramas, sendo simples, mostrou-se limitado para etiquetagem, servindo como base conceitual para a Parte B.

O modelo HMM alcançou bons resultados mesmo com recursos restritos e sem atingir a distância desejada, validando a implementação do algoritmo de *Baum-Welch* e do decodificador de *Viterbi*. Ainda assim, o tempo de convergência foi elevado e a estabilidade numérica exigiu ajustes nos parâmetros de ε .

Como trabalhos futuros, seria interessante implementar vetorização parcial com bibliotecas otimizadas (por exemplo, PyTorch ou JAX), por exemplo.

Conclusão: O modelo HMM proposto atingiu desempenho satisfatório no corpus de teste, confirmando a viabilidade da etiquetagem morfofssintática supervisionada via algoritmos probabilísticos clássicos, mesmo em ambientes com recursos limitados.